

Question 1: (WATER JUG PROBLEM)

Output:

```
(0, 3): [(4, 3), (0, 0), (3, 0)]
(4, 3): [(0, 3), (4, 0)]
(1, 3): [(4, 3), (0, 3), (1, 0), (4, 0)]
(3, 0): [(4, 0), (3, 3), (0, 0), (0, 3)]
(1, 0): [(4, 0), (1, 3), (0, 0), (0, 1)]
(3, 3): [(4, 3), (0, 3), (3, 0), (4, 2)]
(0, 1): [(4, 1), (0, 3), (0, 0), (1, 0)]
(4, 2): [(4, 3), (0, 2), (4, 0), (3, 3)]
(4, 1): [(4, 3), (0, 1), (4, 0), (2, 3)]
(0, 2): [(4, 2), (0, 3), (0, 0), (2, 0)]
(2, 3): [(4, 3), (0, 3), (2, 0), (4, 1)]
(2, 0): [(4, 0), (2, 3), (0, 0), (0, 2)]
USING BFS:
GOAL HAS BEEN FOUND AT THE 14th STEP!
[(0, 0), (4, 0), (0, 3), (4, 3), (1, 3), (3, 0), (1, 0), (3, 3), (0, 1), (4, 2), (4, 1), (0, 2), (2, 3), (2, 0)]
USING DFS:
GOAL HAS BEEN FOUND AT THE 9th STEP!
[(0, 0), (4, 0), (4, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]
```

Code:

```
class Stack(object):
```

```
    def __init__(self):
```

```
        self.arr = []
```

```
    def isEmpty(self):
```

```
        if(len(self.arr) == 0):
```

```
            return True
```

```
        else:
```

```
            return False
```

```
    def push(self,item):
```

```
        self.arr.append(item)
```

```
def pop(self):  
    if(self.isEmpty()):  
        return []  
    else:  
        out = self.arr.pop()  
        return out
```

```
def peek(self):  
    if(self.isEmpty()):  
        return []  
    else:  
        return self.arr[len(self.arr)-1]
```

```
class Queue(object):  
    def __init__(self):  
        self.arr = []  
  
    def isEmpty(self):  
        if(len(self.arr) == 0):  
            return True  
        else:  
            return False
```

```
def enqueue(self, item):
```

```
    self.arr.append(item)
```

```
def dequeue(self):
```

```
    if(not self.isEmpty()):
```

```
        temp = self.arr[0]
```

```
        self.arr.remove(self.arr[0])
```

```
        return temp
```

```
    else:
```

```
        return []
```

```
def peek(self):
```

```
    if(not self.isEmpty()):
```

```
        return self.arr[0]
```

```
    else:
```

```
        return []
```

```
def prepend(list1, a):
```

```
    list2 = [a]
```

```
    list2.append(list1)
```

```
    list1 = list2
```

```
return list1
```

```
class WaterJugAgent(object):
```

```
    def __init__(self, initialState, goalState, capacityOfJugs, stateSpace):
```

```
        self.initialState = initialState
```

```
        self.goalState = goalState
```

```
        self.capacity = capacityOfJugs
```

```
        self.stateSpace = stateSpace
```

```
        self.stateSpace[initialState] = []
```

```
    def stateSpaceRepresentation(self):
```

```
        queue1 = Queue()
```

```
        queue1.enqueue(initialState)
```

```
        visited = []
```

```
        while(not queue1.isEmpty()):
```

```
            currentStateee = queue1.dequeue()
```

```
            if(currentStateee not in visited):
```

```
                visited.append(currentStateee)
```

```
                self.stateSpace[currentStateee] = []
```

```
            sensedData = self.sense(currentStateee)
```

```
            decision = self.decide(sensedData)
```

```
self.actuate(decision, currentStateee)
```

```
adjQueue = Queue()
```

```
for i in self.stateSpace[currentStateee]:
```

```
    adjQueue.enqueue(i)
```

```
while(not adjQueue.isEmpty()):
```

```
    nextOut = adjQueue.peek()
```

```
    if(nextOut in visited):
```

```
        adjQueue.dequeue()
```

```
    else:
```

```
        queue1.enqueue(adjQueue.dequeue())
```

```
print("STATE SPACE REPERESNTATION: ")
```

```
for i, j in self.stateSpace.items():
```

```
    print(f"{i}: {j}")
```

```
def BFS(self):
```

```
    displayList = []
```

```
    queue1 = Queue()
```

```
    queue1.enqueue(initialState)
```

```
    visited = []
```

```
    count = 0
```

```
while(not queue1.isEmpty()):

    currentStateee = queue1.dequeue()

    if(currentStateee not in visited):

        visited.append(currentStateee)

        displayList.append(currentStateee)

        count = count + 1

        if(currentStateee == self.goalState):

            print(f"GOAL HAS BEEN FOUND AT THE {count}th STEP!")

adjQueue = Queue()

for i in self.stateSpace[currentStateee]:

    adjQueue.enqueue(i)

while(not adjQueue.isEmpty()):

    nextOut = adjQueue.peek()

    if(nextOut in visited):

        adjQueue.dequeue()

    else:

        queue1.enqueue(adjQueue.dequeue())

print(displayList)
```

```
def DFS(self):

    displayList = []

    stack1 = Stack()

    stack1.push(initialState)

    visited = []

    count = 0

    while(not stack1.isEmpty()):

        currentStateee = stack1.pop()

        if(currentStateee not in visited):

            visited.append(currentStateee)

            displayList.append(currentStateee)

            count = count + 1

            if(currentStateee == self.goalState):

                print(f"GOAL HAS BEEN FOUND AT THE {count}th STEP!")

                break

        adjQueue = Queue()

        temp = self.stateSpace[currentStateee]

        temp.reverse()

        for i in temp:

            adjQueue.enqueue(i)
```

```
while(not adjQueue.isEmpty()):  
    nextOut = adjQueue.peek()  
    if(nextOut in visited):  
        adjQueue.dequeue()  
  
    else:  
        stack1.push(adjQueue.dequeue())  
  
print(displayList)
```

```
def sense(self, currentState):  
    sensedStates = currentState  
    return sensedStates  
  
def decide(self, sensedStates):  
    decisions = []  
  
    if(sensedStates[0]<self.capacity[0]):  
        calculateNext = (self.capacity[0],sensedStates[1])  
        if calculateNext != sensedStates:  
            decisions.append(f"fill {self.capacity[0]} litre jug")
```



```
if(sensedStates[1]<self.capacity[1]):  
    calculateNext = (sensedStates[0], self.capacity[1])  
    if calculateNext != sensedStates:  
        decisions.append(f"fill {self.capacity[1]} litre jug")
```

```
if(sensedStates[0]>0):  
    calculateNext = (0,sensedStates[1])  
    if calculateNext != sensedStates:  
        decisions.append(f"drop {self.capacity[0]} litre jug on ground")
```

```
if(sensedStates[1]>0):  
    calculateNext = (sensedStates[0],0)  
    if calculateNext != sensedStates:  
        decisions.append(f"drop {self.capacity[1]} litre jug on ground")
```

```
if(sensedStates[0]+sensedStates[1]>0 and sensedStates[0]+sensedStates[1]>self.capacity[0] and  
sensedStates[1]>0):
```

```
    calculateNext = (self.capacity[0],sensedStates[1]-(self.capacity[0]-sensedStates[0]))  
    if calculateNext != sensedStates:  
        decisions.append(f"pour {self.capacity[1]} litre jug to {self.capacity[0]} litre jug")
```

```
if(sensedStates[0]+sensedStates[1]>0 and sensedStates[0]+sensedStates[1]>self.capacity[1] and  
sensedStates[0]>0):
```

```
    calculateNext = (sensedStates[0]-(self.capacity[1]-sensedStates[1]), self.capacity[1])
```

```

    if calculateNext != sensedStates:

        decisions.append(f"pour {self.capacity[0]} litre jug to {self.capacity[1]} litre jug")

    if(sensedStates[0]+sensedStates[1]>0 and sensedStates[0]+sensedStates[1]<=self.capacity[0] and
sensedStates[1]>=0):

        calculateNext = (sensedStates[0] + sensedStates[1],0)

        if calculateNext != sensedStates:

            decisions.append(f"pour all water from {self.capacity[1]} litre jug to {self.capacity[0]} litre jug")

    if(sensedStates[0]+sensedStates[1]>0 and sensedStates[0]+sensedStates[1]<=self.capacity[1] and
sensedStates[0]>=0):

        calculateNext = (0,sensedStates[0] + sensedStates[1])

        if calculateNext != sensedStates:

            decisions.append(f"pour all water from {self.capacity[0]} litre jug to {self.capacity[1]} litre jug")

    return decisions

```

```

def actuate(self, decisions, currentState):

    if(f"fill {self.capacity[0]} litre jug" in decisions):

        action = (self.capacity[0],currentState[1])

    if action != currentState:

        self.stateSpace[currentState].append(action)

```

```
if(f"fill {self.capacity[1]} litre jug" in decisions):
```

```
    action = (currentState[0], self.capacity[1])
```

```
    if action != currentState:
```

```
        self.stateSpace[currentState].append(action)
```

```
if(f"drop {self.capacity[0]} litre jug on ground" in decisions):
```

```
    action = (0,currentState[1])
```

```
    if action != currentState:
```

```
        self.stateSpace[currentState].append(action)
```

```
if(f"drop {self.capacity[1]} litre jug on ground" in decisions):
```

```
    action = (currentState[0],0)
```

```
    if action != currentState:
```

```
        self.stateSpace[currentState].append(action)
```

```
if(f"pour {self.capacity[1]} litre jug to {self.capacity[0]} litre jug" in decisions):
```

```
    action = (self.capacity[0],currentState[1]-(self.capacity[0]-currentState[0]))
```

```
    if action != currentState:
```

```
        self.stateSpace[currentState].append(action)
```

```
if(f"pour {self.capacity[0]} litre jug to {self.capacity[1]} litre jug" in decisions):
```

```
    action = (currentState[0]-(self.capacity[1]-currentState[1]), self.capacity[1])
```

```
    if action != currentState:
```

```
        self.stateSpace[currentState].append(action)
```

```
if(f"pour all water from {self.capacity[1]} litre jug to {self.capacity[0]} litre jug" in decisions):
```

```
    action = (currentState[0] + currentState[1],0)
```

```
    if action != currentState:
```

```
        self.stateSpace[currentState].append(action)
```

```
if(f"pour all water from {self.capacity[0]} litre jug to {self.capacity[1]} litre jug" in decisions):
```

```
    action = (0,currentState[0] + currentState[1])
```

```
    if action != currentState:
```

```
self.stateSpace[currentState].append(action)
```

```
jug1 = 0#int(input("ENTER THE INITIAL STATE OF THE FIRST JUG: "))
```

```
jug2 = 0#int(input("ENTER THE INITIAL STATE OF THE SECOND JUG: "))
```

```
initialState = (jug1, jug2)
```

```
capacity1 = 4#int(input("ENTER THE CAPACITY OF JUG 1:"))
```

```
capacity2 = 3#int(input("ENTER THE CAPACITY OF JUG 2:"))
```

```
capacity = (capacity1, capacity2)
```

```
goal1 = 2#int(input("ENTER THE GOAL STATE OF JUG 1:"))
```

```
goal2 = 0#int(input("ENTER THE GOAL STATE OF JUG 2:"))
```

```
goal_state = (goal1, goal2)
```

```
agent = WaterJugAgent(initialState, goal_state, capacity, {})
```

```
agent.stateSpaceRepresentation()
```

```
print("USING BFS:")
```

```
agent.BFS()
```

```
print("USING DFS:")
```

```
agent.DFS()
```